```
Name : Farukh Hasan Shaha
Div: SY-B
Roll No. : B-35
Branch: AI&D
```

Aim: Use sequence data types and their associated operations in Python a) Strings b) Lists c) Arrays d) Tuples e) Dictionary f) Sets g) Range Theory: 1.what are the sequence data types in python ? name them and explain how indexing and slicing work across different sequence types ? Ans : sequence are ordered collection where every item has specific position . types : list , tuple, sting, range. Indexing : indexing start from 0 and -1 is the last . slicing : [start:stop:step] allows us to extract sub section. 2.what is the difference between list and tuple in python ? why would you prefer tuple over a list in real world application ? Ans : List []: list mutable . use for collection of items that will grow or change . Tuple (): tuple is imutable . use for fixe data like coordinates(x,y) or setting . They are faster and safer for for data integrity. 3.How are python array different from lists ? when should you use the array module instead of a list? Ans : List : can store diferent types (int ,float, string) together . Thay are flexible but use more memory . Array : can only store one type (eg., all integer). use case : use array only when processing millions of numeric value where memory efficiency is critical . 4.explain how dictionary work interanally in python . why are dictionary lokups faster compared to list or tuples ? Ans: Dictionary use hash tables . the process : a key is converted into a "hash" ,which points directly to a specific memory address. speed : Because it jumps straight to the address , lookups are O(1) . In a list , you have to research item -by item(O(n)), which gets slower as the list grows . 5.what is the difference between sets and list in python ? how do sets handle duplicate values , and where sets useful in practcal scenarios ? Ans : sets {}: unordered and unique .they automatically dalate duplicates . Lists; ordered and allow duplicates . practical use: sets are best for "memebership testing " and mathematical operations like finding common items between two groups

```
import array

# String Example
s ="Hello Python"
print("Length:", len(s))
print("Upper:", s.upper())
print("Lower:", s.lower())
print("Replace:", s.replace("Python", "World"))
print("Slice:", s[0:5])

#list operation
l = [10, 20, 30, 40]
l.append(50)
print("Append:", l)
l.insert(1, 15)
print("Insert:", l)
l.remove(30)
print("Remove:", l)
l.sort()
print("Sort:", l)
print("Length:", len(l))

#array operation
arr = array.array('i', [1, 2, 3, 4])
arr.append(5)
print("Append:", arr)
arr.insert(1, 10)
print("Insert:", arr)
arr.remove(3)
print("Remove:", arr)
arr.reverse()
print("Reverse:", arr)
print("Index of 10:", arr.index(10))

# Tuple Example
t = (1, 2, 3, 4, 2)
print("Length:", len(t))
print("Count of 2:", t.count(2))
print("Index of 3:", t.index(3))
print("Slice:", t[1:3])
t2 = t +(5, 6)
print("Concatenation:", t2)

# Dictionary Example
d = {"name": "Krushna", "age": 2}
d["city"] = "Pune"
print("Add:", d)
d["age"] = 21
print("Update:", d)
d.pop("city")
print("Remove:", d)
print("Keys:", d.keys())
print("Values:", d.values())
```

```
# Set Example
s = {1, 2, 3, 4}
s.add(5)
print("Add:", s)
s.remove(2)
print("Remove:", s)
s2 = {4, 5, 6}
print("Union:", s.union(s2))
print("Intersection:", s.intersection(s2))
print("Length:", len(s))

# Range Example
r = range(1, 10)
print("Range:", list(r))
r2 = range(1, 10, 2)
print("Step:", list(r2))
print("Length:", len(r))
print("Element at index 3:", r[3])
print("List:", list(r))
```

```
Length: 12
Upper: HELLO PYTHON
Lower: hello python
Replace: Hello World
Slice: Hello
Append: [10, 20, 30, 40, 50]
Insert: [10, 15, 20, 30, 40, 50]
Remove: [10, 15, 20, 40, 50]
Sort: [10, 15, 20, 40, 50]
Length: 5
Append: array('i', [1, 2, 3, 4, 5])
Insert: array('i', [1, 10, 2, 3, 4, 5])
Remove: array('i', [1, 10, 2, 4, 5])
Reverse: array('i', [5, 4, 2, 10, 1])
Index of 10: 3
Length: 5
Count of 2: 2
Index of 3: 2
Slice: (2, 3)
Concatenation: (1, 2, 3, 4, 2, 5, 6)
Add: {'name': 'Krushna', 'age': 2, 'city': 'Pune'}
Update: {'name': 'Krushna', 'age': 21, 'city': 'Pune'}
Remove: {'name': 'Krushna', 'age': 21}
Keys: dict_keys(['name', 'age'])
Values: dict_values(['Krushna', 21])
Add: {1, 2, 3, 4, 5}
Remove: {1, 3, 4, 5}
Union: {1, 3, 4, 5, 6}
Intersection: {4, 5}
Length: 4
Range: [1, 2, 3, 4, 5, 6, 7, 8, 9]
Step: [1, 3, 5, 7, 9]
Length: 9
Element at index 3: 4
List: [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Conclusion : In this experiment . different sequence data types in python such as string , list, array, tuples, dictionary ,sets and range were implemented . various basic operation were performed on each data type . this experiment helped in understanding hoe sequence data types work and how they are used in pytho